Бие даалт №1

Хэсэгчлэх ба дахин дуудах арга

Алгоритмын зохиомж ба шинжилгээ (F.CSM301)

A төвшин · Python, Java

Оюутан: Б. Билгүүн (**B190910014**)

Багш: Д. Батмөнх

ШУТИС, МХТС · Улаанбаатар, 2026

Агуулга

1 Ерөнхий мэдээлэл **3**

1.1 Ажлын зорилго . . . 3

1.2 Хэсэгчлэх арга ба дахин дуудах тэгшитгэл . . . 3

1.3 Файлын бүтэц . . . 3

1.4 Тестийн ерөнхий үр дүн . . . 3

1.5 Хэмжилтийн орчин ба арга . . . 4

2 Бодлого 1. Majority Element (LeetCode 169) **4**

2.1 Зорилго . . . 4

2.2 Алгоритмын санаа . . . 4

2.3 Хэрэгжүүлэлт . . . 5

2.4 Тестийн үр дүн . . . 5

2.5 Хугацаа . . . 5

2.6 Санах ой . . . 5

3 Бодлого 2. Convert Sorted Array to Binary Search Tree (LeetCode 108) **6**

3.1 Зорилго . . . 6

3.2 Алгоритмын санаа . . . 6

3.3 Хэрэгжүүлэлт . . . 6

3.4 Тестийн үр дүн . . . 6

3.5 Хугацаа . . . 6

3.6 Санах ой . . . 7

4 Бодлого 3. Longest Nice Substring (LeetCode 1763) **7**

4.1 Зорилго . . . 7

4.2 Алгоритмын санаа . . . 7

4.3 Хэрэгжүүлэлт . . . 7

4.4 Тестийн үр дүн . . . 7

4.5 Хугацаа . . . 8

4.6 Санах ой . . . 8

5 Бодлого 4. Different Ways to Add Parentheses (LeetCode 241) **8**

5.1 Зорилго . . . 8

5.2 Алгоритмын санаа (B төвшин) . . . 8

5.3 Хугацаа ба санах ой (B төвшин) . . . 9

5.4 Сайжруулсан шийдэл (A төвшин) . . . 9

6 Бодлого 5. Construct Binary Tree from Preorder and Inorder Traversal (LeetCode 105) **10**

6.1 Зорилго . . . 10

6.2 Алгоритмын санаа (B төвшин) . . . 10

6.3 Хугацаа ба санах ой (B төвшин) . . . 11

6.4 Сайжруулсан шийдэл (A төвшин) 11

7 Дүгнэлт **12**

A Java хэрэгжүүлэлт **13**

A.1 1. Majority Element (`MajorityElement.java`) 13

A.2 2. Sorted Array → BST (`SortedArrayToBst.java`) 13

A.3 3. Longest Nice Substring (`LongestNiceSubstring.java`) 13

A.4 4. Different Ways — анхны (`DifferentWays.java`) 14

A.5 4. Different Ways — сайжруулсан (`DifferentWaysImproved.java`) 15

A.6 5. Build Tree — анхны (`BuildTree.java`) 16

A.7 5. Build Tree — сайжруулсан (`BuildTreeImproved.java`) 16

1 Ерөнхий мэдээлэл

1.1 Ажлын зорилго

Хэсэгчлэх аргаар бодлогыг дэд бодлогод хувааж, дэд бодлогыг дахин дуудалтаар бодож, шийдүүдийг нэгтгэх зарчмыг хэрэгжүүлэх. А төвшний шаардлагын дагуу таван бодлогыг бүгдийг **Python** болон **Java** дээр бодож, 4 ба 5-р бодлогын гүйцэтгэлийг **сайжруулав**. Бодлого бүрийн хугацаа, санах ойн зарцуулалтыг онолоор тооцоолж, бодит хэмжилтээр шалгав.

№	Бодлого	Төвшин	Арга
1	Majority Element (LeetCode 169)	C	хэсэгчлэх
2	Convert Sorted Array to Binary Search Tree (LeetCode 108)	C	дахин дуудах
3	Longest Nice Substring (LeetCode 1763)	C	хэсэгчлэх
4	Different Ways to Add Parentheses (LeetCode 241)	B → A	хэсэгчлэх → товъёг + санах (memo)
5	Construct Binary Tree from Preorder and Inorder Traversal (LeetCode 105)	B → A	дахин дуудах → толь + товъёг

1.2 Хэсэгчлэх арга ба дахин дуудах тэгшитгэл

Хэсэгчлэх арга гурван алхамтай: **хуваах** (ижил төрлийн жижиг дэд бодлогууд), **бодох** (дэд бодлогыг дахин дуудалтаар; хангалттай жижиг бол шууд), **нэгтгэх** (дэд шийдүүдээс эх бодлогын шийд). n хэмжээтэй бодлогыг n/b хэмжээтэй a дэд бодлогод хувааж, хуваах ба нэгтгэхэд f(n) ажил зарцуулбал

$$T(n) = a \cdot T(n/b) + f(n).$$

Ерөнхий теоремоор (лекцийн Теорем 5.1–5.2) f(n)-г $n^{\log_b a}$ урсгал функцтэй харьцуулна: (1) $f = O(n^{\log_b a - \varepsilon}) \to \Theta(n^{\log_b a})$; (2) $f = \Theta(n^{\log_b a} \lg^k n)$, $k \geq 0 \to \Theta(n^{\log_b a} \lg^{k+1} n)$; (3) $f = \Omega(n^{\log_b a + \varepsilon})$ ба $a \cdot f(n/b) \leq c \cdot f(n)$, $c < 1 \to \Theta(f(n))$. Тэгш биш хуваалттай (жишээ нь 5-р бодлого) тэгшитгэлийг давталтын мод эсвэл орлуулгаар бодно.

1.3 Файлын бүтэц

```
prj1/B190910014/
├── python/src/          p1_majority_element.py ... p5_build_tree_improved.py
├── python/tests/        test_problems.py (pytest)
├── python/bench.py      хэмжилт
├── java/src/main/...    MajorityElement.java ... BuildTreeImproved.java, Bench.java
├── java/src/test/...    ProblemsTest.java (JUnit 5)
├── report/              prj1_report.tex, prj1_report.pdf, make_tables.py, build.sh
└── env_info.sh
```

LeetCode-д илгээхдээ Python-д `class Solution`-ийг шууд, Java-д классын нэрийг `Solution` болгож (`package` мөрийг хасаж) илгээнэ. `TreeNode`-ийг LeetCode өөрөө өгдөг.

1.4 Тестийн ерөнхий үр дүн

Бодлого бүрийг LeetCode-ийн жишээ болон **шууд аргатай (brute force) харьцуулсан санамсаргүй тохиолдлоор** шалгав:

Бодлого	Шалгасан зүйл
1	300 санамсаргүй массив, `Counter`/`groupingBy`-ээр тоолсон хариутай
2	n = 0, 1, 2, 3, 7, 8, 100, 1000: inorder нь эрэмбэлэгдсэн (хайлтын мод), өндөр яг $\lceil \lg(n+1) \rceil$
3	2000 санамсаргүй мөр, бүх дэд мөрийг шалгах $\Theta(n^3)$ аргатай
4	k = 0…8 (тус бүр 5 илэрхийлэл): хоёр хувилбарын хариу ижил, хариуны тоо C_k, дуудалт 3^k ба $(k+1)(k+2)/2$
5	200 санамсаргүй мод + 2000 зангилаатай гинж: сэргээсэн модны preorder, inorder анхныхтай ижил

Хэл	Команд	Үр дүн
Python	`cd python && .venv/bin/pytest -v`	28 passed
Java	`cd java && mvn test`	Tests run: 6, Failures: 0

1.5 Хэмжилтийн орчин ба арга

WSL2 Ubuntu 22.04, Intel Core i5-1334U (12 логик цөм, L2 7.5 MiB, L3 12 MiB), 7.6 GiB RAM, Python 3.10.12, OpenJDK 17.0.20.1 (`env_info.sh`).

- **Хугацаа.** Нэг багц $\geq$ 20 ms байтал дуудалтыг давтаж, нэг дуудалтын дунджийг гаргав. Python-д 3, Java-д 5 багцын медиан. Java-д эхний багцуудаар JIT-ийг халааж, хэмжилт бүрийн өмнө GC ажиллуулав.
- **Санах ой.** Python: `tracemalloc`-ийн **оргил** утга (дуудалтын явцад Python-ы объектуудад нэгэн зэрэг эзэлсэн хамгийн их санах ой). Java: `ThreadMXBean.getThreadAllocatedBytes`-ээр нэг дуудалтад **нийт хуваарилсан** санах ой (GC-ээр чөлөөлөгдсөн ч тоологдоно). Хоёр хэмжигдэхүүн ижил биш тул хэл хооронд шууд харьцуулахгүй, харин n өсөхөд хэрхэн өөрчлөгдөх хандлагыг харьцуулна. Дахин дуудалтын стек аль алинд нь тоологдохгүй.

Хэмжилтийг дахин хийж тайланг шинэчлэх дараалал:

```
cd python && python3 bench.py                        # -> out/results_python.tsv
cd java && java -Xss64m -cp build com.prj1.Bench     # -> out/results_java.tsv
cd report && python3 make_tables.py && ./build.sh    # хүснэгт -> .tex -> PDF
```

2 Бодлого 1. Majority Element (LeetCode 169)

2.1 Зорилго

n урттай массиваас хагасаас олон ($\lfloor n/2 \rfloor + 1$ ба түүнээс дээш) удаа орсон (олонх) гишүүнийг олно. Ийм гишүүн заавал байна.

2.2 Алгоритмын санаа

Ажиглалт: m нь бүх массивын олонх гишүүн бол зүүн эсвэл баруун хагасын **ядаж нэгд нь** олонх байна. Хэрэв хоёр хагасын аль алинд нь хагасаас олон биш байсан бол нийт тоо нь n/2-оос хэтрэхгүй. Тиймээс:

1. **Хуваах:** [lo, hi)-г дундуур нь хоёр хуваана.
2. **Бодох:** хагас бүрийн олонх гишүүнийг дахин дуудалтаар олно (суурь: ганц гишүүн).
3. **Нэгтгэх:** хоёр нэр дэвшигч ижил бол тэр. Өөр бол [lo, hi) мужид тус бүрийг тоолж, олон удаа орсныг сонгоно.

```
def majority(lo, hi):                  # nums[lo:hi]-ийн олонх гишүүн
    if hi - lo == 1:
        return nums[lo]
    mid = (lo + hi) // 2
    left, right = majority(lo, mid), majority(mid, hi)
    if left == right:
        return left
    left_count = sum(1 for i in range(lo, hi) if nums[i] == left)
    right_count = sum(1 for i in range(lo, hi) if nums[i] == right)
    return left if left_count > right_count else right
```

2.3 Хэрэгжүүлэлт

- **Python** (`p1_majority_element.py`): `class Solution` доторх `majority(lo, hi)` функц. Массивыг зүсэхгүй, хил заагийг л дамжуулна.
- **Java** (`MajorityElement.java`): `majority(nums, lo, hi)` арга. Дунд цэгийг `(lo + hi) >> 1` гэж олж, int-ийн халилтаас сэргийлэв. Хоёр нэр дэвшигчийг нэг давталтаар зэрэг тоолно.

2.4 Тестийн үр дүн

- LeetCode-ийн 3 жишээ: [3, 2, 3] → 3, [2, 2, 1, 1, 1, 2, 2] → 2, [7] → 7.
- 300 санамсаргүй массив ($n \le 60$, олонх гишүүнийг зориуд $\lfloor n/2 \rfloor + 1$-ээс олон оруулсан): Python-д `Counter`, Java-д `groupingBy`-ээр тоолсон хариутай бүгд таарсан.

2.5 Хугацаа

Хамгийн муу үед төвшин бүрд тоолох ажил $\Theta(n)$: $\mathbf{T(n) = 2T(n/2) + \Theta(n)} \to a = b = 2$, $n^{\log_2 2} = n = f(n)$, 2-р тохиолдол ($k = 0$) $\to \mathbf{\Theta(n \lg n)}$. Хамгийн сайн үед (бүх гишүүн ижил) нэгтгэх алхам $\Theta(1)$ тул $T(n) = 2T(n/2) + \Theta(1) = \Theta(n)$.

n	Python, ms	Java, ms	Python t/(n lg n), нс	Python санах ой, KB	Java санах ой, KB
1 000	1.126	0.012	113	1.6	0.0
10 000	12.8	0.118	96	2.2	0.0
100 000	123.6	1.572	74	2.5	0.0
1 000 000	1 431	16.0	72	2.8	0.0

n 10 дахин өсөхөд хугацаа $\approx 10 \cdot \lg(10n)/\lg n$ дахин өсөх ёстой. Python-д $10^5 \to 10^6$ үед 11.6 дахин (онол 12.0), Java-д 10.2 дахин өссөн. «t / (n lg n)» багана 113 → 72 нс гэж бага зэрэг буурч, том n дээр тогтворжиж байна. Жижиг n дээр функцийн дуудалт, хэмжилтийн тогтмол зардал харьцангуй их байдаг тул ингэж буурч байгаа нь $\Theta(n \lg n)$ үнэлгээтэй зөрөхгүй.

2.6 Санах ой

Массивыг хуулахгүй, зөвхөн (lo, hi) товъёгоор дамжуулна. Нэмэлт санах ой нь дахин дуудалтын стекийн гүн $\lceil \lg n \rceil$ буюу $\mathbf{\Theta(\lg n)}$. Хэмжилтэд Python-ы оргил хэдхэн KB, Java-д хуваарилалт 0 байгаа нь (стек тоологдохгүй) үүнийг харуулна.

Харьцуулахад Boyer–Moore-ын санал хураах арга нэг удаа гүйгээд $\mathbf{\Theta(n)}$ хугацаа, $\mathbf{\Theta(1)}$ санах ойтой. Хэсэгчлэх арга $\lg n$ дахин удаан ч бодлогыг хуваж, нэгтгэх санааг тодорхой харуулдаг.

3 Бодлого 2. Convert Sorted Array to Binary Search Tree (LeetCode 108)

3.1 Зорилго

Өсөхөөр эрэмбэлэгдсэн массиваас **өндрөөр тэнцвэртэй** хоёртын хайлтын мод байгуулна.

3.2 Алгоритмын санаа

Дунд гишүүнийг үндэс болговол зүүн тал нь бүгд бага, баруун тал нь бүгд их (хайлтын модны нөхцөл), хоёр талын хэмжээ хамгийн ихдээ 1-ээр ялгаатай (тэнцвэр). Хоёр талыг дахин дуудалтаар байгуулна.

```
def build(lo, hi):                    # nums[lo..hi]
    if lo > hi:
        return None
    mid = (lo + hi) // 2
    return TreeNode(nums[mid], build(lo, mid - 1), build(mid + 1, hi))
```

Модны өндөр яг $\lceil \lg(n+1) \rceil$ бөгөөд энэ нь n зангилаатай хоёртын модны хамгийн бага өндөр (тестээр шалгав).

3.3 Хэрэгжүүлэлт

- **Python** (`p2_sorted_array_to_bst.py`), **Java** (`SortedArrayToBst.java`): `build(lo, hi)` нь `TreeNode(nums[mid], build(lo, mid − 1), build(mid + 1, hi))`-ийг буцаана. `TreeNode`-ийг LeetCode өгдөг. Өөрийн компьютер дээр `tree_node.py`, `TreeNode.java`-г ашиглана.

3.4 Тестийн үр дүн

n = 0, 1, 2, 3, 7, 8, 100, 1000 бүрд хоёр хэл дээр: (1) модны inorder тойролт анхны массивтай ижил, өөрөөр хэлбэл хайлтын мод; (2) зүүн, баруун дэд модны өндрийн зөрүү ≤ 1; (3) өндөр яг $\lceil \lg(n+1) \rceil$. Бүгд давсан.

3.5 Хугацаа

Дуудалт бүр $\Theta(1)$ ажил хийж хоёр хагаст дуудна: $\mathbf{T(n) = 2T(n/2) + \Theta(1) \to n^{\log_2 2} = n \gg 1 = O(n^{1-\varepsilon})}$, $\varepsilon = 1$, 1-р тохиолдол $\to \mathbf{\Theta(n)}$. Зангилаа бүрийг яг нэг удаа үүсгэдэг тул доод хязгаар $\Omega(n)$ бөгөөд түүнээс бага байж чадахгүй.

n	Python, ms	Java, ms	Python санах ой, KB	Java санах ой, KB	Java, байт/ зангилаа
1 000	0.662	0.040	143.9	23.5	24
10 000	12.4	0.116	1 480	234.4	24
100 000	187.8	1.015	14 840	2 344	24
1 000 000	6 825	42.7	148 435	23 438	24

Java-д $n = 10^4 \to 10^5$ үед хугацаа 8.7 дахин өссөн. $10^5 \to 10^6$ үед 42 дахин өссөн нь 23 MB шинэ объект үүсгэхэд GC ажилласантай холбоотой. Python-д 10^6 зангилаа үүсгэхэд санах ой хуваарилалт, GC давамгайлж шугамаас хурдан өссөн. Гэхдээ хийж буй **алхмын тоо** яг n дуудалт тул онолын эрэмбэ $\Theta(n)$ хэвээр.

3.6 Санах ой

Гаралт өөрөө n зангилаа тул $\Theta(\mathbf{n})$, үүнээс гадна дахин дуудалтын стек $\Theta(\lg n)$. Java-д зангилаа бүр яг 24 байт (объектын толгой 12 + int 4 + хоёр заагч 2×4, 8-д тэгшилсэн) эзэлж байгаа нь хүснэгтийн сүүлийн баганаас харагдана. Python-ы объект илүү том ($\approx$ 150 байт/зангилаа).

4 Бодлого 3. Longest Nice Substring (LeetCode 1763)

4.1 Зорилго

Үсэг бүр нь том, жижиг хоёр хэлбэрээрээ орсон мөрийг «сайхан» гэнэ. Хамгийн урт сайхан дэд мөрийг олно. Ижил урттай хэд байвал хамгийн эхнийхийг буцаана.

4.2 Алгоритмын санаа

«Муу» үсэг: эсрэг хэлбэр нь мөрөнд огт байхгүй үсэг. Муу үсэг агуулсан дэд мөр хэзээ ч сайхан болохгүй. Тиймээс:

1. **Хуваах:** мөрийг бүх муу үсгээр нь нэг дор хэсэглэнэ.
2. **Бодох:** хэсэг бүрийг дахин дуудалтаар бодно.
3. **Нэгтгэх:** хамгийн урт хариуг авна (тэнцүү бол эхнийх).

Муу үсэг байхгүй бол s өөрөө сайхан.

```
def longestNiceSubstring(s):
    if len(s) < 2:
        return ""
    present = set(s)
    best, start, split = "", 0, False
    for i, ch in enumerate(s):
        if ch.swapcase() not in present:          # муу үсэг
            split = True
            part = longestNiceSubstring(s[start:i])
            if len(part) > len(best):
                best = part
            start = i + 1
    if not split:
        return s
    part = longestNiceSubstring(s[start:])
    return part if len(part) > len(best) else best
```

4.3 Хэрэгжүүлэлт

- **Python** (`p3_longest_nice_substring.py`): `present = set(s)`, хэсгийг зүсэж (`s[start:i]`) дахин дуудна.
- **Java** (`LongestNiceSubstring.java`): мөрийг хуулахгүй, `solve(s, lo, hi)` товъёгоор дамжуулна. Үсэг ASCII тул олонлогийн оронд `boolean[128]` массив хэрэглэнэ. Эцэст нь ганц удаа `substring` хийнэ.

4.4 Тестийн үр дүн

- LeetCode-ийн 4 жишээ: "YazaAay"→ "aAa", "Bb"→ "Bb", "c"→ , "dDzeE"→ "dD".
- «aAbBcC» үсгүүдээс бүрдсэн 0–14 урттай 2000 санамсаргүй мөр: бүх дэд мөрийг шалгадаг $\Theta(n^3)$ шууд аргатай (ижил урттай бол эхнийх) таарсан.

4.5 Хугацаа

Хэсгүүдийн нийт урт $< n$ тул төвшин бүрийн нийт ажил $\Theta(n)$: $T(n) = \Sigma\, T(n_i) + \Theta(n)$, $\Sigma\, n_i < n$. **Гүний хязгаар:** нэг төвшинд хуваасан муу үсгийн төрөл хүүхдийн бүх хэсгээс бүрмөсөн алга болно. Тиймээс төвшин бүрд хэсэгт байгаа үсгийн төрлийн тоо дор хаяж 1-ээр буурч, гүн $\le |\Sigma| = 52$ болно. Нийт $\Theta(|\Sigma| \cdot \mathbf{n}) = \Theta(\mathbf{n})$ ($|\Sigma|$ тогтмол).

Хэрэв зөвхөн эхний муу үсгээр хуваавал «aaaa…a» мэт оролтод гүн n болж $\Theta(n^2)$ болно. Бүх муу үсгээр нэг дор хуваах нь энэ муу тохиолдлоос сэргийлдэг.

n	Оролт	Python, ms	Java, ms	Python t/n, нс	Java t/n, нс	Python санах ой, KB	Java санах ой, KB
100	nice	0.019	0.001	190	10	0.9	0.2
100	split	0.081	0.008	810	80	3.6	3.5
1 000	nice	0.232	0.004	232	4	1.0	0.2
1 000	split	0.741	0.029	741	29	3.7	27.6
10 000	nice	2.193	0.084	219	8	1.0	0.2
10 000	split	8.143	0.800	814	80	4.4	307.4
100 000	nice	35.0	1.310	350	13	1.0	0.2
100 000	split	131.5	6.169	1315	62	4.6	2 902

«nice» оролтод мөр бүхэлдээ сайхан тул ганц удаа гүйж дуусна. «split» оролтод хос хэлбэргүй «z» үсгээр олон хуваалт хийгдэж, дахин дуудалт ажиллана. Аль алинд нь «t / n» тогтмол орчим байгаа нь $\Theta(n)$-ийг баталж байна.

4.6 Санах ой

`present` олонлог $\Theta(|\Sigma|)$. Python хувилбар дэд мөрийг хуулдаг тул нэг төвшинд $\Theta(n)$ түр санах ой, стекийн гүн ≤ 52. Java хувилбар хуулалгүй (lo, hi) товъёгоор дамжуулж, дуудалт бүрд `boolean[128]` хэрэглэдэг. Тиймээс **нэгэн зэрэг амьд** байх нэмэлт санах ой нь $\Theta(|\Sigma| \cdot \text{гүн}) = \mathbf{\Theta(1)}$ (гаралтын мөрөөс бусдаар). Python-ы оргил утга оролтын уртаас үл хамааран ≈ 1–5 KB байгаа нь үүнийг харуулж байна.

Java-гийн хүснэгтэд «split» үед санах ой n өсөхийн хэрээр өсөж байгаа нь **нийт хуваарилалт** юм. Дуудалт бүр шинэ `boolean[128]` (≈ 144 байт) үүсгэх бөгөөд «z» үсэг 11 тэмдэгт тутамд нэг таарч, дуудалтын тоо ≈ n/11 болно: $n = 10^5$ үед ≈ 9 000 дуудалт × 144 байт + бусад ≈ 2.9 MB. Эдгээр массив дуудалт дуусмагц хог болдог тул оргил санах ойд нөлөөлөхгүй. «Нийт хуваарилалт» ба «оргил санах ой» гэдэг хоёр өөр хэмжигдэхүүнийг ялгах нь чухал.

5 Бодлого 4. Different Ways to Add Parentheses (LeetCode 241)

5.1 Зорилго

`+`, `-`, `*` үйлдэл ба тоо агуулсан илэрхийлэлд хаалтыг бүх боломжит аргаар тавьж гарах бүх утгыг буцаана.

5.2 Алгоритмын санаа (B төвшин)

Хамгийн сүүлд хийгдэх үйлдлийг сонгоод илэрхийллийг хоёр хуваана. Зүүн, баруун талын бүх утгыг дахин дуудалтаар олоод, хос бүрд тэр үйлдлийг хэрэглэнэ.

```
def diffWaysToCompute(expression):
    results = []
    for i, ch in enumerate(expression):
        if ch in "+-*":
```

```
            left = diffWaysToCompute(expression[:i])
            right = diffWaysToCompute(expression[i + 1:])
            for x in left:
                for y in right:
                    results.append(apply(x, y, ch))
    if not results:                         # үйлдэлгүй - ганц тоо
        results.append(int(expression))
    return results
```

5.3 Хугацаа ба санах ой (B төвшин)

k үйлдэлтэй илэрхийлэлд хариуны тоо нь **Каталаны тоо** $C_k = (2k)! \,/\, ((k+1)!\cdot k!) \approx 4^k \,/\, (k^{1.5}\cdot\sqrt{\pi})$. Гаралт өөрөө ийм хэмжээтэй тул аль ч алгоритм $\Omega(C_k)$.

Дуудалтын тоо: $C(0) = 1$, $C(k) = 1 + \Sigma_{j=0}{}^{k-1}\,[C(j) + C(k-1-j)] = 1 + 2\cdot\Sigma_{j=0}{}^{k-1}\,C(j)$ тул $C(k) = 3\cdot C(k-1)$ буюу $\mathbf{C(k) = 3^k}$ (тестээр шалгав). Гэтэл өөр өөр дэд илэрхийлэл ердөө $(k+1)(k+2)/2$ л бий. Тэгэхээр **ижил дэд бодлогыг олон дахин** бодож, дэд мөрийг хуулж, тоог дахин задалж байна. Түр жагсаалтуудын нийт хэмжээ ч гурвын зэргээр өснө.

5.4 Сайжруулсан шийдэл (A төвшин)

1. Илэрхийллийг **нэг удаа** тоо, үйлдлийн жагсаалт болгон задална (дэд мөр хуулахгүй).
2. Дэд бодлогыг операндын **товъёгийн муж (i, j)**-ээр тодорхойлж, хариуг **санана (memo)**.

```
def compute(i, j):                          # nums[i..j] операндууд
    if (i, j) in memo:
        return memo[(i, j)]
    if i == j:
        result = [nums[i]]
    else:
        result = [apply(x, y, ops[k])
                  for k in range(i, j)
                  for x in compute(i, k)
                  for y in compute(k + 1, j)]
    memo[(i, j)] = result
    return result
```

Хэрэгжүүлэлт. Анхны: `p4_different_ways.py`, `DifferentWays.java`. Сайжруулсан: `p4_different_ways_improved.py`, `DifferentWaysImproved.java`. Python-д memo нь `dict`, түлхүүр нь (i, j). Java-д `List<Integer>[][]` хоёр хэмжээст массив, түлхүүр тооцоолох шаардлагагүй тул хурдан. Хоёр хувилбар `calls` тоолуураар бодсон дэд бодлогын тоог бүртгэнэ.

Тестийн үр дүн. LeetCode-ийн жишээ "2-1-1"→ [0, 2], "2*3-4*5"→ [−34, −14, −10, −10, 10] хоёр хувилбарт таарсан. $k = 0\ldots8$ үйлдэлтэй (тус бүр 5) санамсаргүй илэрхийлэлд хоёр хувилбарын хариу ижил, хариуны тоо яг C_k, дуудалтын тоо яг 3^k ба $(k+1)(k+2)/2$ (Python 7, Java 2 тест).

Дэд бодлого бүр яг нэг удаа бодогдоно: $\mathbf{(k+1)(k+2)/2}$ удаа. Хугацаа нь бүх дэд бодлогын хариуг байгуулах ажилтай тэнцүү, ойролцоогоор $\Theta(\Sigma_m\,(k+1-m)\cdot C_m) = \Theta(C_k)$. Энэ нь гаралтын хэмжээтэй ижил эрэмбэтэй тул асимптот оновчтой.

		Дуудалтын тоо		Python, ms		Java, ms		
k	Хариу (Каталан)	анхны	сайжруулсан	анхны	сайжруулсан	анхны	сайжруулсан	Python-д хурдсалт
2	2	9	6	0.012	0.017	0.002	0.004	0.7×
3	5	27	10	0.038	0.034	0.005	0.003	1.1×
4	14	81	15	0.089	0.033	0.014	0.003	2.7×
5	42	243	21	0.377	0.145	0.017	0.002	2.6×
6	132	729	28	0.978	0.139	0.044	0.005	7.0×
7	429	2 187	36	1.985	0.324	0.142	0.016	6.1×
8	1 430	6 561	45	7.464	0.982	0.456	0.046	7.6×
9	4 862	19 683	55	17.8	3.654	1.418	0.193	4.9×
10	16 796	59 049	66	59.9	9.266	4.369	0.498	6.5×

Дуудалтын тоо 3^k байсныг (k+1)(k+2)/2 болгож бууруулсан (k = 10: 59 049 → 66). Хугацаа k = 10 үед Python-д **6.5 дахин**, Java-д **8.8 дахин** хурдан болов. Хугацааны хурдсалт дуудалтын тооны харьцаа шиг (895 дахин) их биш. Учир нь хоёр хувилбарын аль аль нь C_k хэмжээтэй гаралтыг байгуулах ёстой бөгөөд энэ ажил сайжруулсан хувилбарын хугацааны ихэнхийг эзэлдэг. Сайжруулалт **давхардсан** ажлыг арилгасан, харин **гаралтын** ажлыг арилгах боломжгүй.

Санах ой. memo нь бүх (i, j) мужийн хариуг зэрэг хадгалдаг тул Python-ы **оргил** санах ой сайжруулсан хувилбарт бага зэрэг их байна (k = 10: 460 KB → 619 KB). Энэ бол хугацаа ба санах ойн солилцоо (time–space tradeoff). Харин Java-д нэг дуудалтад **нийт хуваарилсан** санах ой анхны хувилбарт 3^k-аар өсөж, сайжруулсанд хэд дахин бага байна:

	Python, KB		Java, KB	
k	анхны	сайжруулсан	анхны	сайжруулсан
2	0.7	1.5	1.4	1.4
3	1.0	2.1	4.4	2.2
4	1.7	3.5	13.3	2.2
5	2.6	5.4	32.7	4.2
6	7.0	14.3	103.4	10.8
7	20.4	37.1	318.3	30.2
8	67.1	111.4	982.8	94.1
9	225.6	342.9	3 060	307.2
10	460.4	618.5	9 415	1 025

6 Бодлого 5. Construct Binary Tree from Preorder and Inorder Traversal (LeetCode 105)

6.1 Зорилго

Ялгаатай утгатай хоёртын модны preorder (үндэс → зүүн → баруун) ба inorder (зүүн → үндэс → баруун) тойролтоос модыг сэргээнэ.

6.2 Алгоритмын санаа (B төвшин)

preorder[0] нь үндэс. inorder-т үндсээс зүүн талд зүүн дэд мод, баруун талд баруун дэд мод байрлана. Хэрэв үндэс inorder-ийн k байрлалд байвал preorder[1..k] нь зүүн дэд модны preorder. Хоёр жагсаалтыг хуваагаад хоёр дэд модыг дахин дуудалтаар сэргээнэ.

```
def buildTree(preorder, inorder):
    if not preorder:
        return None
    root = TreeNode(preorder[0])
```

```
    k = inorder.index(preorder[0])                    # Θ(n) хайлт
    root.left = buildTree(preorder[1:k + 1], inorder[:k])
    root.right = buildTree(preorder[k + 1:], inorder[k + 1:])
    return root
```

6.3 Хугацаа ба санах ой (B төвшин)

Дуудалт бүр шугаман хайлт ба хуулалтад $\Theta(n)$ зарцуулна: $T(n) = T(k) + T(n-1-k) + \Theta(n)$.

- **Тэнцвэртэй ойролцоо мод** ($k \approx n/2$): $T(n) = 2T(n/2) + \Theta(n) = \mathbf{\Theta(n \lg n)}$ (R1).
- **Нэг талдаа тулсан мод** ($k = n-1$ буюу гинж): $T(n) = T(n-1) + \Theta(n) = \Sigma\, \Theta(i) = \mathbf{\Theta(n^2)}$.

Санах ой: зүсмэл бүр шинэ жагсаалт. Гинжин модонд дахин дуудалтын зам дээрх бүх зүсмэл зэрэг амьд байх тул оргил санах ой $n + (n-1) + \ldots \approx n^2/2$ буюу $\mathbf{\Theta(n^2)}$ болно.

6.4 Сайжруулсан шийдэл (A төвшин)

1. inorder-ийн **утга → товъёг** хосуудыг тольд (Python-д `dict`, Java-д `HashMap`) нэг удаа хадгална: хайлт $\mathbf{\Theta(1)}$.
2. Жагсаалт хуулахгүй, inorder-ийн (lo, hi) мужаар дамжуулна. preorder-ийг нэг заагчаар дараалаар уншина, учир нь preorder-т үндэс → зүүн дэд мод → баруун дэд мод гэсэн дараалалтай.

```
position = {value: i for i, value in enumerate(inorder)}
next_root = 0

def build(lo, hi):                                    # inorder[lo..hi]
    nonlocal next_root
    if lo > hi:
        return None
    value = preorder[next_root]; next_root += 1
    k = position[value]                               # Θ(1)
    root = TreeNode(value)
    root.left = build(lo, k - 1)
    root.right = build(k + 1, hi)
    return root
```

Хэрэгжүүлэлт. Анхны: `p5_build_tree.py`, `BuildTree.java` (`Arrays.copyOfRange`-ээр хуулна). Сайжруулсан: `p5_build_tree_improved.py`, `BuildTreeImproved.java` (`HashMap<Integer, Integer>`, `nextRoot` талбар). Гинжин модонд дахин дуудалтын гүн n хүрэх тул Python-д `sys.setrecursionlimit(20000)`, Java-гийн хэмжилтэд `-Xss64m` тохиргоо хэрэглэв.

Тестийн үр дүн. LeetCode-ийн жишээ (preorder [3, 9, 20, 15, 7], inorder [9, 3, 15, 20, 7]), 0–60 зангилаатай 200 санамсаргүй мод болон 2000 зангилаатай гинж дээр сэргээсэн модны preorder, inorder тойролт анхныхтай ижил. Хоёр хувилбар, хоёр хэл дээр давсан.

Одоо $T(n) = T(k) + T(n-1-k) + \Theta(1)$ тул модны хэлбэрээс үл хамааран $\mathbf{\Theta(n)}$. Санах ой: толь $\Theta(n)$ + стек $\Theta(h)$, $h \le n$.

Санамсаргүй мод:

	Python, ms		Java, ms		Python, KB		Java, KB	
n	**анхны**	**сайжруулсан**	**анхны**	**сайжруулсан**	**анхны**	**сайжруулсан**	**анхны**	**сайжруулсан**
500	1.567	0.592	0.108	0.542	87.5	94.5	81.2	56.9
1 000	1.678	1.222	0.066	0.053	172.0	200.4	169.1	115.7
2 000	4.927	2.308	0.132	0.183	328.6	418.4	364.2	233.3
4 000	6.605	5.714	0.278	0.380	613.5	835.9	760.5	468.4

Санамсаргүй модны өндөр $\approx O(\lg n)$ тул анхны хувилбар дунджаар $\Theta(n \lg n)$ бөгөөд Python-ы зүсэлт, `index` нь C хэл дээр хэрэгжсэн тул хурдан. Хоёр хувилбарын ялгаа бага, хэдхэн мс-ийн төвшинд хэмжилтийн хэлбэлзэл ч нөлөөлж байна. Сайжруулалтын жинхэнэ ач холбогдол хамгийн муу тохиолдолд харагдана.

Гинж (хамгийн муу тохиолдол):

	Python, ms		Java, ms		Python, KB		Java, KB	
n	**анхны**	**сайжруулсан**	**анхны**	**сайжруулсан**	**анхны**	**сайжруулсан**	**анхны**	**сайжруулсан**
500	3.535	0.629	0.322	0.046	2 211	241.6	1 020	52.9
1 000	14.5	1.346	0.973	0.054	8 436	595.5	3 992	111.7
2 000	64.7	2.762	6.794	0.100	32 604	1 303	15 797	229.3
4 000	269.4	6.314	53.7	0.289	127 815	2 719	62 844	464.4

7 Дүгнэлт

№	**Бодлого**	**Дахин дуудах тэгшитгэл**	**Хугацаа**	**Нэмэлт санах ой**
1	Majority Element	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \lg n)$	$\Theta(\lg n)$
2	Sorted Array → BST	$T(n) = 2T(n/2) + \Theta(1)$	$\Theta(n)$	$\Theta(n)$ гаралт + $\Theta(\lg n)$
3	Longest Nice Substring	$T(n) = \Sigma T(n_i) + \Theta(n)$, гүн ≤ 52	$\Theta(n)$	$\Theta(1)$ (Java)
4	Different Ways — анхны	$C(k) = 1 + 2\Sigma C(j) = 3^k$ дуудалт	3^k дахин бодолт	давхардсан жагсаалт
4	Different Ways — сайжруулсан	$(k+1)(k+2)/2$ дэд бодлого	$\Theta(C_k)$ (гаралттай ижил)	memo
5	Build Tree — анхны	$T(n) = T(k) + T(n-1-k) + \Theta(n)$	$\Theta(n \lg n) \ldots \Theta(n^2)$	$\Theta(n^2)$ хүртэл
5	Build Tree — сайжруулсан	$T(n) = T(k) + T(n-1-k) + \Theta(1)$	$\Theta(n)$	$\Theta(n)$

Гинжин модонд n хоёр дахин өсөхөд анхны хувилбарын санах ой яг 4 дахин (Java 15.8 → 62.8 MB), хугацаа Python-д 4.2 дахин (64.7 → 269 ms) өсөж, $\mathbf{\Theta(n^2)}$-ийг баталж байна. Java-д хугацаа 7.9 дахин (6.8 → 53.7 ms) өссөн нь n^2 хэмжээтэй (62 MB) түр массивууд кэшид багтахгүй, GC-ийн ачаалал нэмэгдсэнтэй холбоотой. Сайжруулсан хувилбарт хугацаа $\approx$ 2–3 дахин буюу $\mathbf{\Theta(n)}$. $n = 4000$ үед хугацаа Python-д **43 дахин**, Java-д **186 дахин**, санах ой Python-д **47 дахин**, Java-д **135 дахин** багассан.

Гол дүгнэлтүүд:

1. **Хэсэгчлэх аргын өртөг нь хуваах, нэгтгэх алхмын өртгөөс шууд хамаарна.** 1, 2-р бодлого хоёулаа хоёр хагаст хуваадаг ч нэгтгэх алхам $\Theta(n)$ ба $\Theta(1)$ байгаагаас хариу нь $\Theta(n \lg n)$ ба $\Theta(n)$ болж ялгаатай гарсан.
2. **Дэд бодлого давхцаж байвал санах (memo).** 4-р бодлогод ижил дэд илэрхийллийг 3^k удаа бодож байсныг $(k+1)(k+2)/2$ болгов. Энэ нь динамик программчлалын санаа бөгөөд Лаб 4-ийн сэдэв.
3. **Хуулалт ба шугаман хайлтыг товъёг, толиор солих.** 5-р бодлогод хамгийн муу тохиолдлын $\Theta(n^2)$-ийг $\Theta(n)$ болгосон. Санамсаргүй өгөгдөл дээр ялгаа бага ч хамгийн муу оролтод хэдэн арав дахин.
4. **Хуваах стратеги хамгийн муу тохиолдлыг тодорхойлно.** 3-р бодлогод бүх муу үсгээр нэг дор хуваах нь гүнийг ≤ 52 болгож, $\Theta(n^2)$ болох муу тохиолдлоос сэргийлсэн.
5. **Python, Java хоёрын үр дүнгийн хандлага ижил, тогтмол үржүүлэгч өөр.** Java 15–160 дахин хурдан байсан ч (P4: 14×, P1: 89×, P2: 160×) n өсөхөд өсөх хурд хоёр хэлэнд ижил байв. Санах ойн хэмжилтэд «оргил» ба «нийт хуваарилалт» гэсэн хоёр өөр хэмжигдэхүүнийг ялгаж тайлбарлах хэрэгтэй.

A Java хэрэгжүүлэлт

Python кодыг бодлого бүрийн «Алгоритмын санаа» хэсэгт үзүүлсэн. Доор Java шийдлүүдийг `java/src/main/java/com/prj1/` хавтаснаас бүтнээр нь оруулав. LeetCode-д илгээхдээ `package` мөрийг хасаж, классын нэрийг `Solution` болгоно.

A.1 1. Majority Element (`MajorityElement.java`)

```
package com.prj1;

/** LeetCode 169. Хэсэгчлэх арга: T(n) = 2T(n/2) + Θ(n) = Θ(n lg n), нэмэлт санах ой Θ(lg n). */
public class MajorityElement {

    public int majorityElement(int[] nums) {
        return majority(nums, 0, nums.length);
    }

    /** nums[lo, hi)-ийн олонх гишүүн. */
    private int majority(int[] nums, int lo, int hi) {
        if (hi - lo == 1) return nums[lo];
        int mid = (lo + hi) >>> 1;
        int left = majority(nums, lo, mid);
        int right = majority(nums, mid, hi);
        if (left == right) return left;
        // Олонх гишүүн аль нэг хагаст заавал олонх байна - хоёр нэр дэвшигчийг тоолно.
        int leftCount = 0;
        int rightCount = 0;
        for (int i = lo; i < hi; i++) {
            if (nums[i] == left) leftCount++;
            else if (nums[i] == right) rightCount++;
        }
        return leftCount > rightCount ? left : right;
    }
}
```

A.2 2. Sorted Array → BST (`SortedArrayToBst.java`)

```
package com.prj1;

/** LeetCode 108. Дунд гишүүнийг үндэс болгоно: T(n) = 2T(n/2) + Θ(1) = Θ(n). */
public class SortedArrayToBst {

    public TreeNode sortedArrayToBST(int[] nums) {
        return build(nums, 0, nums.length - 1);
    }

    /** nums[lo..hi]-аас тэнцвэртэй хайлтын мод. */
    private TreeNode build(int[] nums, int lo, int hi) {
        if (lo > hi) return null;
        int mid = (lo + hi) >>> 1;
        return new TreeNode(nums[mid], build(nums, lo, mid - 1), build(nums, mid + 1, hi));
    }
}
```

A.3 3. Longest Nice Substring (`LongestNiceSubstring.java`)

```
package com.prj1;

/**
```

```
 * LeetCode 1763. Хэсэгчлэх арга: эсрэг томын хэлбэр нь мөрөнд байхгүй «муу» үсгүүдээр нэг дор
 * хуваана. Хэсэг бүрээс дор хаяж нэг үсгийн төрөл алга болдог тул гүн ≤ 52, нийт Θ(n).
 */
public class LongestNiceSubstring {

    public String longestNiceSubstring(String s) {
        return solve(s, 0, s.length());
    }

    /** s[lo, hi) дахь хамгийн урт сайхан дэд мөр. */
    private String solve(String s, int lo, int hi) {
        if (hi - lo < 2) return "";
        boolean[] present = new boolean[128];
        for (int i = lo; i < hi; i++) present[s.charAt(i)] = true;

        String best = "";
        int start = lo;
        boolean split = false;
        for (int i = lo; i < hi; i++) {
            if (!present[swapCase(s.charAt(i))]) { // муу үсэг
                split = true;
                String part = solve(s, start, i);
                if (part.length() > best.length()) best = part; // тэнцүү бол эхнийх
                start = i + 1;
            }
        }
        if (!split) return s.substring(lo, hi);
        String part = solve(s, start, hi);
        return part.length() > best.length() ? part : best;
    }

    private static char swapCase(char c) {
        return Character.isUpperCase(c) ? Character.toLowerCase(c) : Character.toUpperCase(c);
    }
}
```

A.4 4. Different Ways — анхны (`DifferentWays.java`)

```
package com.prj1;

import java.util.ArrayList;
import java.util.List;

/** LeetCode 241 (B төвшин). Үйлдэл бүрээр хоёр хуваж, хоёр талын бүх хариуг хослуулна. */
public class DifferentWays {

    /** Дахин дуудалтын тоо (хэмжилтэд). */
    public long calls = 0;

    public List<Integer> diffWaysToCompute(String expression) {
        calls++;
        List<Integer> results = new ArrayList<>();
        for (int i = 0; i < expression.length(); i++) {
            char ch = expression.charAt(i);
            if (ch == '+' || ch == '-' || ch == '*') {
                List<Integer> left = diffWaysToCompute(expression.substring(0, i));
                List<Integer> right = diffWaysToCompute(expression.substring(i + 1));
                for (int x : left) {
                    for (int y : right) results.add(apply(x, y, ch));
                }
            }
        }
        if (results.isEmpty()) results.add(Integer.parseInt(expression));
        return results;
```

```
    }

    static int apply(int x, int y, char op) {
        return op == '+' ? x + y : op == '-' ? x - y : x * y;
    }
}
```

A.5 4. Different Ways — сайжруулсан (DifferentWaysImproved.java)

```
package com.prj1;

import java.util.ArrayList;
import java.util.List;

/**
 * LeetCode 241 - сайжруулсан (A төвшин). Илэрхийллийг нэг удаа задалж, дэд бодлогыг операндын
 * товъёгийн муж (i, j)-ээр тодорхойлж, хариуг санана. Өөр дэд бодлого (k+1)(k+2)/2.
 */
public class DifferentWaysImproved {

    /** memo-д байхгүй үед бодсон дэд бодлогын тоо (хэмжилтэд). */
    public long calls = 0;

    private int[] nums;
    private char[] ops;
    private List<Integer>[][] memo;

    @SuppressWarnings("unchecked")
    public List<Integer> diffWaysToCompute(String expression) {
        List<Integer> numList = new ArrayList<>();
        StringBuilder opList = new StringBuilder();
        int value = 0;
        for (char ch : expression.toCharArray()) {
            if (Character.isDigit(ch)) {
                value = value * 10 + (ch - '0');
            } else {
                numList.add(value);
                opList.append(ch);
                value = 0;
            }
        }
        numList.add(value);
        nums = numList.stream().mapToInt(Integer::intValue).toArray();
        ops = opList.toString().toCharArray();
        memo = new List[nums.length][nums.length];
        return compute(0, nums.length - 1);
    }

    /** nums[i..j] операндуудад хаалт тавих бүх аргын утга. */
    private List<Integer> compute(int i, int j) {
        if (memo[i][j] != null) return memo[i][j];
        calls++;
        List<Integer> result = new ArrayList<>();
        if (i == j) {
            result.add(nums[i]);
        } else {
            for (int k = i; k < j; k++) { // ops[k] нь nums[k], nums[k+1]-ийн хооронд
                for (int x : compute(i, k)) {
                    for (int y : compute(k + 1, j)) result.add(DifferentWays.apply(x, y, ops[k]));
                }
            }
        }
        memo[i][j] = result;
        return result;
```

```
    }
}
```

A.6 5. Build Tree — анхны (BuildTree.java)

```
package com.prj1;

import java.util.Arrays;

/**
 * LeetCode 105 (B төвшин). preorder[0] нь үндэс; түүнийг inorder-ооc шугаман хайлтаар олж,
 * массивыг хуулж хуваана. Хамгийн муу үед T(n) = T(n−1) + Θ(n) = Θ(n²) хугацаа, санах ой.
 */
public class BuildTree {

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        if (preorder.length == 0) return null;
        TreeNode root = new TreeNode(preorder[0]);
        int k = 0;
        while (inorder[k] != preorder[0]) k++; // Θ(n) хайлт
        root.left = buildTree(Arrays.copyOfRange(preorder, 1, k + 1), Arrays.copyOfRange(inorder, 0, k));
        root.right = buildTree(Arrays.copyOfRange(preorder, k + 1, preorder.length),
                Arrays.copyOfRange(inorder, k + 1, inorder.length));
        return root;
    }
}
```

A.7 5. Build Tree — сайжруулсан (BuildTreeImproved.java)

```
package com.prj1;

import java.util.HashMap;
import java.util.Map;

/**
 * LeetCode 105 - сайжруулсан (A төвшин). inorder-ийн утга → товъёгийг тольд (HashMap) хадгалж
 * (хайлт Θ(1)), массив хуулахгүй, зөвхөн товъёгоор дамжуулна. Хугацаа, санах ой Θ(n).
 */
public class BuildTreeImproved {

    private int[] preorder;
    private Map<Integer, Integer> position;
    private int nextRoot;

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        this.preorder = preorder;
        this.position = new HashMap<>();
        for (int i = 0; i < inorder.length; i++) position.put(inorder[i], i);
        this.nextRoot = 0;
        return build(0, inorder.length - 1);
    }

    /** inorder[lo..hi]-ийн мод. preorder-т зүүн дэд мод түрүүлж ирдэг. */
    private TreeNode build(int lo, int hi) {
        if (lo > hi) return null;
        int value = preorder[nextRoot++];
        int k = position.get(value);
        TreeNode root = new TreeNode(value);
        root.left = build(lo, k - 1);
        root.right = build(k + 1, hi);
        return root;
```

```
    }
}
```